



Sri Lanka Institute of Information Technology

Tuition Class Management System Project Report

Information Systems Project 2026

Project ID: ISP_G21

Submitted by:

1. IT24101073 – Praghmarathna M.A.
2. IT24100906 – Pinnedoowe R.D.
3. IT24101204 – Peiris H.D.T.
4. IT24101104 – Attanayake H.M.
5. IT24101767 – Hewage D.N.K

Submitted to:

Ms. Naduni Ranathunga

28/05/2026

Abstract

The Tuition Class Management System is a website that helps manage things like student registration and course enrollment in a tuition class. Normally people do these tasks by hand which take time and can be hard to keep track of. This can also lead to mistakes and misunderstandings. The goal of this project is to create a system that is easy to use and helps both teachers and students manage these tasks better.

The system has five parts: Student Management, Course and Enrollment Management, Class and Attendance Management, Examination and Academic Performance Management and Academic Communication and Activity Management. We figured out what these parts should be at the beginning of the project to make sure the system would work well for a tuition class. The project is mainly about reducing the amount of work people have to do by hand, making sure information is accurate and creating a way for teachers and students to communicate.

The system has a lot of features including registration for students, managing courses and classes scheduling timetables keeping track of attendance scheduling exams, taking exams online managing marks, generating progress reports, making announcements, sending messages and logging activities. By putting in all these features the project helps make academic and administrative tasks easier and more accessible for everyone involved. The system was built as a solution and was thoroughly tested to make sure it meets the requirements and goals of the project. The Tuition Class Management System is a tool that helps teachers and students manage their work efficiently.

Acknowledgement

We would like to express our sincere gratitude to Ms. Narmada Gamage, Lecturer in Charge of the IE2091 Information Systems Project module, for her guidance, support and coordination throughout the module. Her instructions and direction helped us understand the project requirements and complete the work in a structured manner.

We would also like to thank our project supervisor, Ms. Naduni Ranathunga, Instructor, for her continuous support, valuable feedback, and guidance throughout the development of our project. Her advice helped us improve our work and complete each stage of the project successfully.

We are also grateful to the Sri Lanka Institute of Information Technology (SLIIT) and the academic staff of the IE2091 Information Systems Project module for providing us with the opportunity to carry out this project and apply our knowledge in a practical environment.

Furthermore, we would like to thank all members of our project group for their dedication, teamwork, cooperation, and commitment throughout the development of the Tuition Class Management System. Each member contributed their ideas, skills, and effort from the beginning to the completion of the project.

Finally, we would like to express our appreciation to everyone who supported us directly or indirectly during the completion of this project. Their encouragement and assistance helped us complete our work successfully.

Declaration

We declare that this project report or part of it was not a copy of a document done by any organization, university any other institute or a previous student project group at SLIIT and was not copied from the Internet or other sources.

Project Details

Project Title	Tuition Class Management System
Project ID	ISP_G21

Group Members

Reg. No	Name	Signature
IT24101073	Praghnarathna M.A.	
IT24100906	Pinnedoowe R.D	
IT24101104	Attanayake H.M.	
	Peiris H.D.T.	
IT24101767	Hewage D.N.K.	

Table of Contents

Abstract.....	1
Acknowledgement	2
Declaration	3
Table of Contents	4
List of Figures	5
List of Tables	5
List of Acronyms and Abbreviations	6
1.Introduction.....	8
1.1 Problem Statement.....	8
1.2 Product Scope.....	9
1.3 Project Report Structure.....	11
2. Methodology	12
2.1 Requirements and Analysis	12
2.2 Design	16
2.3 Implementation.....	17
2.4 Testing	19
2.4.1 Test Plan	19
2.4.2 Non-Functional Requirements Testing	20
2.4.3 Test Case Summary	21
2.4.4 Traceability to Acceptance Criteria	21
3. Evaluation	24
3.1 Assessment of the Project results	24
3.2 Lessons Learned	27
3.3 Future Work	29
4. Conclusion	32
5.References	35
Appendix A: Design Diagrams	36
Appendix B: Test Results	52
Appendix C: Selected Code Listings	56

List of Figures

Figure 1: UML Class Diagram.....	36
Figure 2: Approve Student Registration Request	37
Figure 3: Approve or Reject Payment Slips	38
Figure 4: Teacher Marks Attendance	39
Figure 5: Conduct Online Exam.....	40
Figure 6: Approve Student Registration Request	41
Figure 7: Approve or Reject Monthly Payment Slips	42
Figure 8: Teacher Marks Attendance	43
Figure 9: Conduct Online Exam.....	44
Figure 10: State diagram.....	45
Figure 11: Student Management	46
Figure 12: Course and Enrollment Management.....	47
Figure 13: Class and Attendance Management.....	48
Figure 14: Examination and Academic Performance Management	49
Figure 15: Academic Communication & Activity Management.....	50
Figure 16: ER Diagram	51
Figure 17: Student Registration Status Interface	53
Figure 18: Course Management Interface	54
Figure 19: Timetable Session Creation Interface.....	54
Figure 20: Exam Scheduling Interface	55
Figure 21: Announcement Creation Interface.....	55
Figure 22: Student Approval and ID Generation	56
Figure 23: Student Approval and ID Generation	57
Figure 24: Request Enrollment.....	58
Figure 25: Timetable Conflict Detection Algorithm.....	59
Figure 26: Exam Submission Marking	60
Figure 27: Announcement Creation	61

List of Tables

Table 1: Non-Functional Requirements.....	15
Table 2: Constraints	16
Table 3: Verified NFRs	21
Table 4: Test Cases	53

List of Acronyms and Abbreviations

Acronym / Abbreviation	Full Form	Meaning in This Project
API	Application Programming Interface	The interface is used for communication between the frontend and backend in the Tuition Class Management System.
CRUD	Create, Read, Update, Delete	The four basic database operations performed across all modules including Student Management, Course Management, Attendance, Examination and Attendance modules.
CSS	Cascading Style Sheets	The styling language used to design the user interface of the Tuition Class Management System frontend components.
DoD	Definition of Done	A set of completion criteria used in the Agile Scrum process. A user story is marked Done only when it is implemented, tested, integrated, documented, and accepted.
DTO	Data Transfer Object	Java classes are used in the Spring Boot backend to transfer data between the controller and service layers without directly exposing entity classes.
ER	Entity-Relationship	The database modelling technique used to define relationships between Tuition Class Management System entities such as Student, Course, Batch, Attendance, Exam, and Payment.
FR	Functional Requirement	A system requirement that specifies what the Tuition Class Management System must do. Twelve functional requirements are defined from FR01 to FR12.
HTML	HyperText Markup Language	The standard markup language used to create the structure of the Tuition Class Management System frontend web pages.
HTTP	HyperText Transfer Protocol	The communication protocol used for all API calls between the React frontend and Spring Boot backend REST endpoints
IDE	Integrated Development Environment	Software tools used for Tuition Class Management System development. IntelliJ IDEA was used for backend development and Visual Studio Code for frontend development.

JPA	Java Persistence API	The standard Java API used via Spring Data JPA and Hibernate ORM to map Java entity classes to database tables and perform database operations.
JWT	JSON Web Token	The stateless authentication token issued by the backend upon successful login. The token contains the user's role and is validated on every protected API request.
NFR	Non-Functional Requirement	System quality attributes defined for the Tuition Class Management System including usability, security, performance, reliability, maintainability, scalability, and compatibility.
ORM	Object-Relational Mapping	The technique used through Hibernate to automatically map Java entity classes such as Student.java, Course.java, and Attendance.java to their corresponding database tables.
REST	Representational State Transfer	The architectural style used to design all backend API endpoints. Endpoints follow patterns like /api/students, /api/courses, and /api/attendance.
SQL	Structured Query Language	The query language used to interact with the Microsoft SQL Server database, executed automatically by Hibernate ORM via Spring Data JPA repository interfaces.
TC	Test Case	A set of conditions used to verify that the system functions correctly. Sixteen test cases from TC01 to TC16 were executed covering all five modules.
UML	Unified Modelling Language	The standard notation used in system design documentation, including Class Diagram, Sequence Diagrams, Activity Diagrams, State Diagram, and Use Case Diagrams.
URL	Uniform Resource Locator	The web address format used to define REST API endpoint paths, for example /api/auth/register, /api/courses, and /api/attendance/mark.
US	User Story	A requirement unit written in the format "As a [role], I want [feature], so that [benefit]". User stories were defined across the five epics of the Tuition Class Management System.

1.Introduction

1.1 Problem Statement

In a tuition class the teacher has to do a lot of things by hand. This includes things like signing up students picking classes making schedules keeping track of who's there planning exams keeping track of grades and checking payments. Usually, these things are done with paper records or different methods that are not organized. As more students join it gets really hard to keep track of everything. It takes a lot of time and mistakes can happen easily. This makes it tough to keep records, find information quickly and make sure the tuition class runs smoothly.

Another big problem is that there is no way for the teacher to talk to the students. Important things like class times, exam dates, updates on schoolwork, announcements and other notices are often told in person or in a way. This can cause confusion when people might miss information and it can be hard to coordinate everything. This affects both the teacher and the students in a way.

These problems show that we need a system that can organize all the information in one place. The Tuition Class Management System was made to help with this. It is a simple and easy to use way to manage student information, school records, exams, communication and other things. The Tuition Class Management System wants to make things easier, for the teacher to make sure information is correct and create an organized place for both the teacher and the students to work together. The Tuition Class Management System is here to help with the tuition class.

1.2 Product Scope

The Tuition Class Management System is a website that helps people manage tuition classes. It is meant to make things easier and more organized for teachers and students. The Tuition Class Management System wants to replace ways of doing things with a new digital system that is better and more accurate.

The Tuition Class Management System does a lot of things. It has five parts. These are managing students, managing courses and enrollments, managing classes and attendance managing exams and grades and managing communication between teachers and students. These five parts are the things that the Tuition Class Management System does. They help tuition classes run smoothly.

The Student Management part of the Tuition Class Management System helps with things like signing up students, checking their information and giving them student IDs. The Course and Enrollment Management part helps teachers make courses manage who is in each course upload materials and check if students have paid. The Class and Attendance Management part helps make schedules check for conflicts, mark attendance and make summaries of who was in class. The Tuition Class Management System is very helpful for managing tuition classes. It makes things easier for teachers and students. The Tuition Class Management System is a tool for tuition classes.

The system has things like Examination and Academic Performance Management. This means you can schedule exams, give exams online, put in marks publish results and make progress reports. It also has Academic Communication and Activity Management. This includes things, like announcements, messages and logs of what people are doing. The system helps teachers and students talk to each other in a way. All these parts work together to help manage the things a tuition class needs to do, like academics and administration for the tuition class.

The product we are talking about is a tuition class management solution. This solution is for one teacher and the students. It helps with things, like work and taking care of student administration. It also helps people talk to each other in a way.

The system is made to help people in ways. For example, it reduces the amount of work people have to do by hand. It also helps people manage information better. The teacher and students can get to services more easily. The system helps the teacher and students work together effectively. The tuition class management solution is what makes all these things possible.

1.3 Project Report Structure

This project report is divided into a part to make it easy to understand how the Tuition Class Management System was developed. The first part, Chapter 1, is about what the project's why we did it. It talks about the problems we were trying to solve and what the system can do. Why we needed to make a digital system to help with schoolwork and other tasks in a tuition class.

The Tuition Class Management System is explained in a way that's easy to follow. We used a template to make sure our report was organized. Chapter 2 is about how we made the Tuition Class Management System. It includes what we needed to do, how we designed it, how we built it and how we tested it. This part of the report talks about what the system can do, how it works, what tools we use to make it and how we make sure it works correctly.

The next part, Chapter 3, is about how the Tuition Class Management System project did. It talks about what we learned, what we could do better and how we can make the system even better in the future. This helps us think about what we did and what we can improve. Then there is Chapter 4, which is the end of the report. It summarizes the project, what we accomplished and what we got out of it.

Finally, Chapter 5 has all the references we used, and the extra parts have things, like diagrams, test results and some of the code we wrote for the Tuition Class Management System.

2. Methodology

2.1 Requirements and Analysis

Requirements gathering and analysis were done extensively by the development team in order to have clear understanding of the problems existing within the manual tuition class management system and also to determine what specific needs that must be met for the project to meet its objective. Some of the critical problems identified include the manual nature of student registration, attendance recording, preparation of timetables, examination handling, payment checking, and general communication. This causes problems such as overload of the tuition teacher, data corruption, data search problems, data redundancy, and miscommunication among the teacher and students.

The primary stakeholder is the Teacher who manages all academic activities within the tuition class. The secondary stakeholders are the students who need easy access to their class timetables, learning materials, examination results, and general academic progress.

The system is structured around five major epics to ensure complete coverage of all essential tuition class activities:

1. Student Management
2. Course and Enrollment Management
3. Class and Attendance Management
4. Examination and Academic Performance Management
5. Academic Communication and Activity Management

Functional Requirements are presented in detail below:

Epic 1 (Student Management)-

- FR1 – The system shall allow students to submit personal details through an online registration form to request access to the system.
- FR2 – The system shall validate mandatory fields before accepting registration data and store the request with a Pending status.
- FR3 – The system shall allow students to log in using their username and password to access the system.
- FR4 – The system shall allow students to view their registration status (Pending, Approved, Rejected) and display the generated student ID if approved or the rejection reason if rejected.
- FR5 – The system shall allow teachers to view and manage student registration requests.
- FR6 – The system shall allow teachers to approve or reject student registrations and automatically generate a unique student ID for approved students.
- FR7 – The system shall allow teachers to update and manage student status (Active, Inactive, Suspended) and record status changes in the student status history.
- FR8 – The system shall allow students to submit feedback about the teacher and allow the teacher to view the submitted feedback.

Epic 2(Course and Enrollment Management)-

- FR9- The system shall allow teachers to create courses based on grade, subject, and batch.
- FR10- The system shall allow teachers to create and manage batches.
- FR11- The system shall allow students to request enrollment in courses.
- FR12- The system shall allow teachers to approve or reject enrollment requests.
- FR13- The system shall assign approved students to relevant batches.
- FR14- The system shall allow teachers to upload subject materials by month.
- FR15- The system shall allow students to upload payment slips.

- FR16- The system shall allow teachers to verify payment slips.
- FR17- The system shall allow teachers to approve or reject payment slips.
- FR18- The system shall restrict access to materials for unpaid students.
- FR19- The system shall allow students to download materials after approval.

Epic 3(Class and Attendance Management)-

- FR20-The system shall allow teachers to create class timetables.
- FR21- The system shall prevent scheduling conflicts in timetables.
- FR22- The system shall allow teachers to view class schedules.
- FR23- The system shall allow students to view their class timetables.
- FR24- The system shall allow teachers to mark attendance for each class session.
- FR25- The system shall allow teachers to update attendance records.
- FR26- The system shall allow students to view their attendance records.
- FR27- The system shall generate monthly attendance summaries.
- FR28- The system shall allow students to view attendance summaries.

Epic 4(Examination and Academic Performance Management)-

- FR29- The system shall allow teachers to create examinations for each subject and grade.
- FR30- The system shall allow teachers to edit examination details.
- FR31- The system shall allow teachers to delete examinations.
- FR32- The system shall allow students to view examination schedules.
- FR33- The system shall allow students to attempt online examinations.
- FR34- The system shall allow students to submit answers for examinations.
- FR35- The system shall store submitted answers securely in the database.
- FR36- The system shall allow teachers to enter and update student marks.
- FR37- The system shall allow teachers to generate student performance reports.
- FR38- The system shall allow teachers to publish examination results.
- FR39- The system shall allow students to view examination results.

Epic 5(Academic Communication and Activity Management)-

- FR40- The system shall allow teachers to create announcements.
- FR41- The system shall allow teachers to edit or delete announcements.
- FR42- The system shall allow students to view announcements.
- FR43- The system shall allow teachers to send messages to students
- FR44- The system shall allow students to send messages to teachers.
- FR45- The system shall store message history for users.
- FR46- The system shall record user activities in the system.
- FR47- The system shall allow teachers to view activity logs.

Non-functional Requirements:

Requirement ID	Category	Description
NFR01	Usability	The system should have a simple, intuitive, and responsive user interface suitable for both teachers and students.
NFR02	Security	Role-based access control using JWT authentication to protect sensitive student and academic data.
NFR03	Performance	System response time shall be less than 2 seconds under normal load.
NFR04	Reliability	The system shall ensure data integrity with proper backup and recovery mechanisms.
NFR05	Maintainability	The system shall follow modular architecture and coding standards for easy future maintenance.
NFR06	Scalability	The system should be capable of supporting up to 500 students.

Table 1: Non-Functional Requirements

Constraints:

Constraint ID	Description
C01	The project must be completed within the given academic semester timeline.
C02	Development must use only free or open-source technologies available to students.
C03	The system is designed exclusively for single-teacher tuition classes.
C04	Full functionality requires stable internet connectivity.
C05	The system must be compatible with modern web browsers (Chrome, Firefox, Edge).

Table 2: Constraints

2.2 Design

The Tuition Class Management System design involved the process of translating the requirements into technical details. The design process was executed through various UML diagrams and database modeling.

The tuition management system follows the three-tier architecture (Presentation Layer, Business Logic Layer, and Data Access Layer), with five modules – Student Management, Course & Enrollment Management, Class and Attendance Management, Examination and Performance Management, and Academic Communications & Activity Management. This modular design allows for seamless interactions among different components of the software.

The Tuition Class Management System has been designed as a web application that supports role-based access control. The teacher will be allowed to access information related to the students, course enrollment, class attendance, examination schedule, and announcements. On the other hand, students will be able to do registration, download course material, view timetable, take examination, and update.

UML Diagrams

The following UML diagrams were created to visualize the system (See Appendix A):

- Use Case Diagram: Illustrates interactions between Teacher and Student actors with the system.
- Class Diagram: Defines the static structure, classes, attributes, and relationships.
- Sequence Diagrams: Shows object interactions for key processes such as registration, enrollment, attendance marking, and examination.
- Activity Diagrams: Depicts workflows and decision points for major operations.
- State Diagram: Represents the lifecycle of entities such as student registration status.

2.3 Implementation

The tuition class management system came up in the implementation stage. It is in this stage that the team came up with a fully functional software application in line with the specifications and designs provided in the previous stages. The software development process was well done to come up with a high-quality end product.

The application system comprises all the components that were recognized during the requirement phase. The components include Student Management, Course & Enrollment Management, Class & Attendance Management, Examination & Academic Performance Management, and Academic Communication & Activities Management. Each module was created for performing its intended functionality, which was then combined to create one platform. This helped make the process of development easier and systematic.

For the front end, technologies such as HTML, CSS, and JavaScript were used for developing an interactive UI. On the other hand, Java programming language and Spring Boot were used for developing the back end. Microsoft SQL Server was chosen for managing the database that will securely store all details including students' details, courses, attendance data, exam results, payments, announcements, messages, and activities performed by users.

Most of the components of the system were developed based on the requirements of the tuition class scenario. This includes the Student Management component, which can be used for online registration with verification, teacher approval process, automatic allocation of Student IDs, and management of student status. Another important component is the Course and Enrollment Management component, which can be used for course creation, batch management, management of enrolment requests, material management, and verification of payment. The other components of the system include the Class and Attendance Management component, which helps in managing timetables, recording of attendance, updating of attendance, and summary reports of attendance.

High importance was accorded with reusable codes in order to reduce redundancy and ensure consistency. This involved validation rules, common user interface layouts, reusable codes at service level, and common database access techniques. All this was consistent with the quality requirements outlined in Sprint 0 that included clean coding, proper naming conventions, and structured coding.

Another critical element that had to be considered while implementing this solution was the validation process, error handling, and consistency of the user interactions. The system features input validation and proper error handling in all its modules. It ensures that no incorrect or incomplete information will be provided and that the system will behave consistently. This had a significant impact on the design of the forms and processing procedures involved.

Integration was another key aspect during the implementation phase. This was because the system was built to be an integrated application, not individual components. Information that was entered in one module could help with tasks in another. This is in line with the Definition of Done created during Sprint 0, where the definition of completion for a feature includes integration.

Important business logic and key code segments are included in Appendix C: Selected Code Listings for reference.

In summary, the implementation stage was successful in translating the theoretical plans and design models of the Tuition Class Management System into an operational system through coding. The frontend was coded using HTML, CSS, and JavaScript while the backend was implemented using Java and Spring Boot. This was done by utilizing the SQL Server Management Studio database in addition to following structured coding guidelines and integrating all modules.

2.4 Testing

2.4.1 Test Plan

The Test Strategy for the Tuition Class Management System has been developed in such a way that all functional requirements (FR1-FR11) and non-functional requirements (NFR1-NFR8) should be verified through testing. For the same reason, the Testing Approach includes testing at various levels, including manual testing of User Interface using browsers and REST API testing using Postman.

The following testing levels were applied:

- Unit Testing – The individual service methods along with queries performed on repositories were tested separately for checking the correctness of the business logic that includes attendance calculation, transition in enroll status, payment slip rules, and mark entry validations.
- Integration Testing – End-to-end testing was done on API endpoints through Postman in order to confirm proper functionality of HTML, CSS frontend, Spring Boot backend, and SQL Server Management Studio database in all the five modules.
- Manual UI Testing – All the processes that can be carried out either by teacher or students were manually executed through Google Chrome, Microsoft Edge, and Firefox in order to validate proper functionality on all browsers (NFR8).

- Boundary and Negative Testing – The cases of invalid input data, student unauthorized access to teacher endpoints, duplicate student registration, conflicting timetable booking, and accessing materials without payment approval were tested for validating robustness and error-handling capabilities of the system.
- Regression Testing – After every sprint, key processes such as login, student registration approval process, enroll, attend marking, and submission of exams were re-tested.

Test evidence, including a test case table and UI screenshots, is provided in Appendix B.

2.4.2 Non-Functional Requirements Testing

In addition to functional testing, the following NFRs were verified:

NFR	Requirement	Verification method	Result
NFR1	Performance – Student dashboard, timetable display, and results page need to load in less than 3 seconds on normal use.	Page loading time analyzed via browser developer tool testing each major page manually	Average page load time ~ 1.8s for all major pages, meeting 3s criterion
NFR2	Security - Passwords should be hashed using bcrypt hashes; students should not have access to teacher-specific functionalities like registration approvals and marks	Inspected bcrypt hashing from AuthService; signed into the account as a student and tried accessing teacher-specific endpoints like approval of registrations, adding marks, and verifying payment slips	Password hashes using bcrypt; student got 403 Forbidden responses for all teacher endpoints
NFR3	Availability – The system is expected to stay accessible during the entire test period with no unexpected shutdowns	Monitored availability of the system during all sprint tests	No unexpected shutdowns were registered throughout the entire course of the project
NFR4	Usability – An inexperienced student should be capable of performing registration and enrollment within 5 minutes	Manual walkthrough test performed by a member of the team timing a first-time user experience	The new student successfully registered and enrolled in classes within 4 minutes, meeting the 5-minute target
NFR5	Scalability – The system should be able to support several concurrent student registrations and enrollments simultaneously.	Simulating concurrent student registrations and course enrollment requests using Postman collection runner for many iterations.	No issues detected during simulations of concurrency.
NFR6	Reliability – System needs to guard against duplicate registration of students, duplication of timetable entry	Submissions for duplicate registration of students; creation of timetable that overlaps existing entry; access of monthly material without payment slip.	All were denied with relevant errors and no data corruption found

	and access of material without a legitimate payment slip.		
NFR7	Maintainability – The programmer should be able to change the module within two workdays without impacting other modules	Changed the Attendance module to include a feature for generating monthly summaries; confirmed that there were no impacts on the Examination and Academic Communication modules.	Change done within one workday with no other side effects observed in other modules
NFR8	Compatibility - The web application needs to be compatible with the current version of Chrome, Edge, and Firefox.	Executed all the processes for students and teachers manually on Chrome 124, Edge 124, and Firefox 125.	All pages worked fine on all three browsers

Table 3: Verified NFRs

2.4.3 Test Case Summary

A total of 16 test cases were executed covering all five modules of the system: Student Management, Course and Enrollment Management, Class and Attendance Management, Examination and Academic Performance Management, and Academic Communication and Activity Management. All test cases passed successfully. The complete test case table including test IDs, modules, test scenarios, expected results, and actual results is provided in Appendix B.

2.4.4 Traceability to Acceptance Criteria

Each functional requirement (FR01 to FR12) was traced to its corresponding acceptance criteria from the Definition of Done established in Sprint 0.

- FR01 (Student Registration) was tested using TC01 and TC02 tests. Students can register by inputting their details, and mandatory fields will be validated prior to storing the registration as Pending.
- FR02 (Approval of Students) was tested using TC03. Teachers have access to view the student's profile and once approved, an automatic generation of student ID takes place with an active status.

- FR03 (Management of Courses/Batches) was tested throughout its implementation process. Teachers have the ability to create batches on the basis of grades, subjects, and the batch year.
- FR04 (Enrollment Requests) and FR05 (Verify Enrollment Requests) were tested using TC04 and TC05. Students are allowed to request enrollments, and teachers are allowed to accept or deny enrollments. The accepted students will be placed in appropriate batches.
- FR06 (Generate Timetable) was tested using TC07. Teachers are allowed to create a timetable for the class, and overlapping classes cannot occur because of a warning message showing a conflict.
- FR07 (Record Attendance) and FR08 (Access Attendance Records) were tested using TC08 and TC09. Teachers are allowed to record accurate attendance, and students are able to access their own attendance records accurately.
- FR09 (Schedule an Exam) and FR10 (Release Exam Results) were tested using TC10 and TC11. Teachers are able to create and conduct exams and then publish exam results that are viewable only by certain students.
- FR11 (Create Announcement) was confirmed with TC13 and TC14. Teachers may post announcements with timestamps, and students may access announcements for the courses they take.
- FR12 (Send Message) was confirmed with TC15 and TC16. Teachers and students may exchange messages, and all messages will be saved in a conversation thread and delivered without fail.

All twelve functional requirements were successfully mapped to their acceptance criteria and tested successfully. The software satisfies the Definition of Done and can now be deployed to an actual tuition class setting.

3. Evaluation

3.1 Assessment of the Project results

This project can be considered a successful one because we managed to address all the issues which we were able to determine at the very outset of the project. Prior to developing the current system, the teacher was required to do all things manually, including student registration, attendance registration, class timetable, exams organization, payment confirmation, as well as communication with students. It should be noted that all these processes took a lot of time and errors occurred quite often.

The developed system has addressed all these issues and brought positive changes to this process. The system provides the efficient management tool that enables the teacher to organize everything connected with teaching process. Furthermore, students get the opportunity to obtain relevant information regarding class timetable, attendance records, exams schedule, results, and notifications. Communication between the teacher and students is also enhanced due to the development of structured communication tools.

The success of the project can be evaluated through the comparison of what was created and what was supposed to be created. In the first sprint, five epics were identified that should be covered in the project: Student Management, Course and Enrollment Management, Class and Attendance Management, Examination and Academic Performance Management, and Academic Communication and Activity Management. All five epics are successfully completed.

Regarding the Student Management epic, its implementation was also successful. Registration of the student takes place on the online platform, and for that purpose, it is necessary to provide one's personal information. The teacher receives requests for registration and accepts or rejects them. Once accepted, a unique student identification number is generated by the system. Also, it is possible to switch the status of the student to such states as Active, Inactive, and Suspended.

The Course and Enrollment Management epic also got a full rollout. Teacher can create courses by grade, subject and batch year. Students can request to register for courses, and the teacher can accept or deny these requests. The teacher can upload subject material for each month, and the students can upload payment slips. The teacher can verify the payment slips and the students can only download the materials for the months when their payment is approved.

Successfully completed the Attendance and Class Management epic. The teacher can prepare class schedules, and the system will warn about time clashes to avoid overlapping sessions. The teacher may take attendance for each class meeting, and attendance records can be updated or corrected if errors are made. Students are able to see their own class timetables and their attendance records. The system also produces monthly attendance summaries for teachers and students.

The Examination and Academic Performance Management epic has been implemented as per plan. The teacher can make exam schedule for each course and each batch. The teacher can also change or delete exams from the schedule. The teacher can enter and edit student marks, view student results and generate progress reports. Then when the results are ready, the teacher can publish the results and students can see their exam results online. Students can also refer to the exam schedules to know their exam dates.

The Academic Communication and Activity Management epic was completely delivered. Teacher can make announcements and students can see them. Teachers and students can communicate via messages, and they both have access to the message history. The system logs user actions so that teachers can view activity logs to see who did what and when. Teachers can also export or backup announcements and activity logs for their records.

In addition to the functional requirements, the system meets the quality standards we established in Sprint 0. The system offers a simple user interface that both teachers and students can utilize. To reduce the amount of incorrect or incomplete data, all modules include proper input validation and error handling. The system has a modular framework,

allowing features to be readily updated or enhanced. Security measures such as limited access to teacher-only features and secure handling of student data have been implemented.

The system's proper operation was demonstrated during the testing phase. All five of the system's modules were covered by a total of sixteen test cases. Every test case was successfully completed. Non-functional needs such as usability, security, performance, dependability, maintainability, scalability, and compatibility were also examined for the system. All of these criteria were satisfied by the system.

All things considered, the Tuition Class Management System effectively accomplishes its primary goals. It lessens the teacher's manual effort, increases the accuracy of student data, and gives teachers and students a structured means of communication. The study shows how a well-thought-out information system may address actual issues in a learning environment. The system is finished and satisfies every requirement specified at the start of the project.

3.2 Lessons Learned

The development of this project taught us key lessons in both technical and project management aspects. One major lesson learned was the importance of carrying out proper requirements analysis before starting implementation.

In the stages of the project, we realized that the existing environment relied heavily on manual processes and unstructured communication. This showed us that a successful software solution must be based on identified user problems and stakeholder needs. Without understanding the problem, it would have been hard to design a system that improves the administrative workflow.

We also learned the value of organizing the system into defined areas. The project was structured around five areas: Student Management, Course and Enrollment Management, Class and Attendance Management, Examination and Academic Performance Management and Academic Communication and Activity Management.

This approach helped us divide the project into sections. We could focus on responsibilities while still contributing to one integrated product. Through this process we learned that modular planning and development make a large project easier to organize, implement, test and maintain.

The project highlighted the importance of backlog planning and task prioritization. In Sprint 0 we identified, prioritized and estimated user stories using story points and hours. This helped us understand which features were essential and which could be developed later. We learned that software development becomes more controlled when there is a plan, a prioritized backlog and a shared understanding of what needs to be completed.

Another lesson learned was the significance of the Definition of Done in maintaining project quality. A feature should be considered complete only when it is implemented, tested, integrated, documented and accepted by stakeholders. This taught us that simply

writing code is not enough. A feature is finished when it works correctly fit into the system and meets quality standards.

From a perspective the project reinforced the importance of validation and error handling. We implemented validation and error handling mechanisms across all modules to minimize data and ensure reliable system behavior. We learned that validation is essential for data accuracy and a smoother user experience.

The project also taught us the importance of usability and interface consistency. The interface needed to be clear, simple and easy to navigate for teachers and students. We realized that poor interface design can create confusion even if the system functions correctly. Software quality is not about technical functionality, but also about user interaction.

We learned another lesson about system integration. The different modules could not be developed as parts. Student data, enrollment information, timetables, examinations, payments and communication features all needed to work. Successful software projects require integration between modules.

The testing phase provided experience and lessons. Testing showed us that we need to verify not whether a feature works in ideal situations but also whether it behaves correctly under different scenarios. Testing should not be treated as a step only but as an ongoing activity.

In addition, the project contributed to our learning in collaboration and project coordination. Effective communication, responsibility sharing and consistency were necessary throughout the project. We learned that good coordination is essential when different members work on components of the same system.

Overall, this project provided lessons in requirements analysis, modular design, planning, quality control, validation, testing, integration, usability and teamwork. These lessons

helped us gain an understanding of how software projects should be planned, developed and evaluated in an academic context. The knowledge and experience gained will be useful for academic work and professional software development projects.

3.3 Future Work

Even though the Tuition Class Management System effectively accomplishes its primary goals and satisfies all of the project's initial requirements, there are a number of improvements that could be implemented in the future to further improve the system. Below is a description of these upcoming enhancements.

Mobile Application: The existing system is made to function as a web application on desktop and laptop computers. It is possible to create a mobile application for both iOS and Android smartphones in the future. Students would be able to use their mobile phones to view their schedules, attendance records, exam schedules, and results. Additionally, it would enable teachers to reply to student messages and record attendance even when they are not using computers.

Better User Interface: Although the existing UI is practical and simple to use, it could be updated and made more aesthetically pleasing. Better color palettes, more user-friendly navigation, and responsive design that functions well on a variety of screen sizes, including tablets and smartphones, are possible features for future iterations of the system.

Automated Notifications: Teachers can now make announcements and send messages using the system, but situations like upcoming tests, awaiting payment verifications, or poor attendance do not automatically trigger notifications. Future iterations may incorporate automated SMS or email alerts to notify teachers when necessary and to remind students of significant dates.

Advanced Reporting and Analytics: The present system produces simple reports like progress reports and monthly attendance summaries. More sophisticated analytics, such as trend analysis of student performance over time, comparison of attendance patterns across courses, and forecasts of student outcomes based on past data, may be included in later iterations.

Online Payment Integration: Teachers can manually validate payment slips that students upload using the present system. In order to enable students to make payments straight through the system, future iterations may link with online payment processors. This would speed up and simplify the procedure for both teachers and students, doing away with the requirement for manual payment verification.

Real-time chat feature: The present messaging system functions similarly to email in that messages are sent and received, but not instantly. Future iterations may incorporate a WebSocket-based real-time chat function that allows instructors and students to converse in real time within the system. This would facilitate faster query resolution and enhance communication.

Export and Backup Enhancements: Teachers can export activity logs and announcements from the existing system. Additional export options, such as exporting student names, attendance information, and exam results to other file formats like Excel, PDF, and CSV, may be included in later editions. To guarantee that data is never lost, automated backup tools can also be used.

Performance Optimization: The system may need to manage more data as the number of students and courses increases. To guarantee that the system stays quick and responsive even under high demand, future iterations may incorporate performance enhancements like database indexing, query optimization, and caching.

Multilingual Support: All user interface text in the current system is written in English. To make the system more accessible to users who feel more at ease speaking in their own tongue, future iterations may include support for multiple languages, such as Tamil and Sinhala.

Since all the essential requirements have been met, the current version of the system does not require these future enhancements. Nonetheless, they are worthwhile improvements that can be included in later versions to further improve the Tuition Class Management System's utility and efficacy for educators and learners.

4. Conclusion

The project for the Tuition Class Management System was successfully completed. This project's primary objective was to develop a digital system that would make it easier for teachers to handle administrative and academic responsibilities while also facilitating students' easier access to information. Prior to the development of this system, the teacher was required to perform numerous tasks by hand, such as student registration, attendance monitoring, scheduling, exam administration, payment verification, and student communication. This manual method required a lot of time, and errors frequently occurred. Additionally, there was a lack of organization in the teacher-student exchange, which frequently resulted in miscommunication.

These issues are resolved by the system we developed. It offers a single platform for managing all administrative and academic tasks. In addition to managing classes and batches, creating timetables with conflict checking, marking attendance, scheduling exams, entering marks, publishing results, making announcements, sending messages, and viewing activity logs, the teacher may also register students, approve or reject registration requests, generate student IDs, and manage courses and batches. Registration, enrollment requests, schedules, attendance records, online tests, results, announcements, messaging, and message history are all available to students.

The project used a methodical approach. To comprehend the issues and determine what the system needed to do, we began with requirements analysis. Next, we used UML diagrams, such as class, sequence, activity, state, and use case diagrams, to develop the system. Java and Spring Boot were used for the backend, HTML, CSS, and JavaScript were used for the frontend, and Microsoft SQL Server was used for the database. Unit testing, integration testing, manual user interface testing, boundary testing, and regression testing were among the several layers of testing that were conducted. All five modules were covered by the 16 test cases that were run, and every test case succeeded.

Every one of the five epics that were specified at the start of the project was fully executed. Teacher can authorize registrations and control student status through student management. Teacher may establish courses and batches, handle enrollment requests, upload resources, and check payment slips with Course and Enrollment Management. While students can examine their schedules and attendance records, class and attendance management enables teacher to make schedules, record attendance, and provide summaries. While students can view test dates and outcomes, teachers can plan exams, enter marks, publish results, and create progress reports using Examination and Academic Performance Management. Teacher can post announcements, send messages, and view activity logs using Academic Communication and Activity Management, and students can view announcements, send messages, and view message history.

Additionally, the system satisfies the first quality standards. The user interface is straightforward and user-friendly. To guarantee data accuracy, error management and input validation are used in every module. The system's modular design facilitates future updates and extensions. Students cannot access teacher-only features thanks to security features including role-based access restriction.

The testing outcomes show that the system functions well. Every functional requirement, ranging from FR01 to FR12, was verified in relation to its acceptance criteria. Usability, security, performance, dependability, maintainability, scalability, and compatibility were all met as non-functional requirements. The system is finished and prepared for usage in an actual classroom setting.

The team also learned a lot from the project. Requirements analysis, modular design, backlog planning, task prioritization, quality control, validation, testing, integration, usability, collaboration, and time management were among the topics we studied. Both our future academic endeavors and our professional jobs will benefit from these teachings.

In conclusion, the Tuition Class Management System effectively accomplishes its goals. It makes academic and administrative chores more effective and well-organized, reduces the

teacher's manual workload, increases the accuracy of student information, and facilitates structured communication between teachers and students. The study shows how a well-thought-out information system may address actual issues in a learning environment. Teacher and students can utilize the system since it satisfies all of the standards that were established at the start of the project.

5.References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [2] S. Pressman, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.
- [3] VMware, "Spring Boot Reference Documentation," [Spring.io](https://docs.spring.io/spring-boot/docs/current/reference/html/). [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. [Accessed: May 20, 2026]
- [4] Microsoft, "SQL Server Technical Documentation," [Microsoft.com](https://learn.microsoft.com/en-us/sql/). [Online]. Available: <https://learn.microsoft.com/en-us/sql/>. [Accessed: May 20, 2026]
- [5] Mozilla Developer Network, "Web Technologies Documentation," MDN Web Docs. [Online]. Available: <https://developer.mozilla.org/>. [Accessed: May 20, 2026]
- [6] SLIIT, "IE2091 - Information Systems Project Module," Sri Lanka Institute of Information Technology, Colombo, Sri Lanka, 2026

Appendix A: Design Diagrams

UML Class Diagram

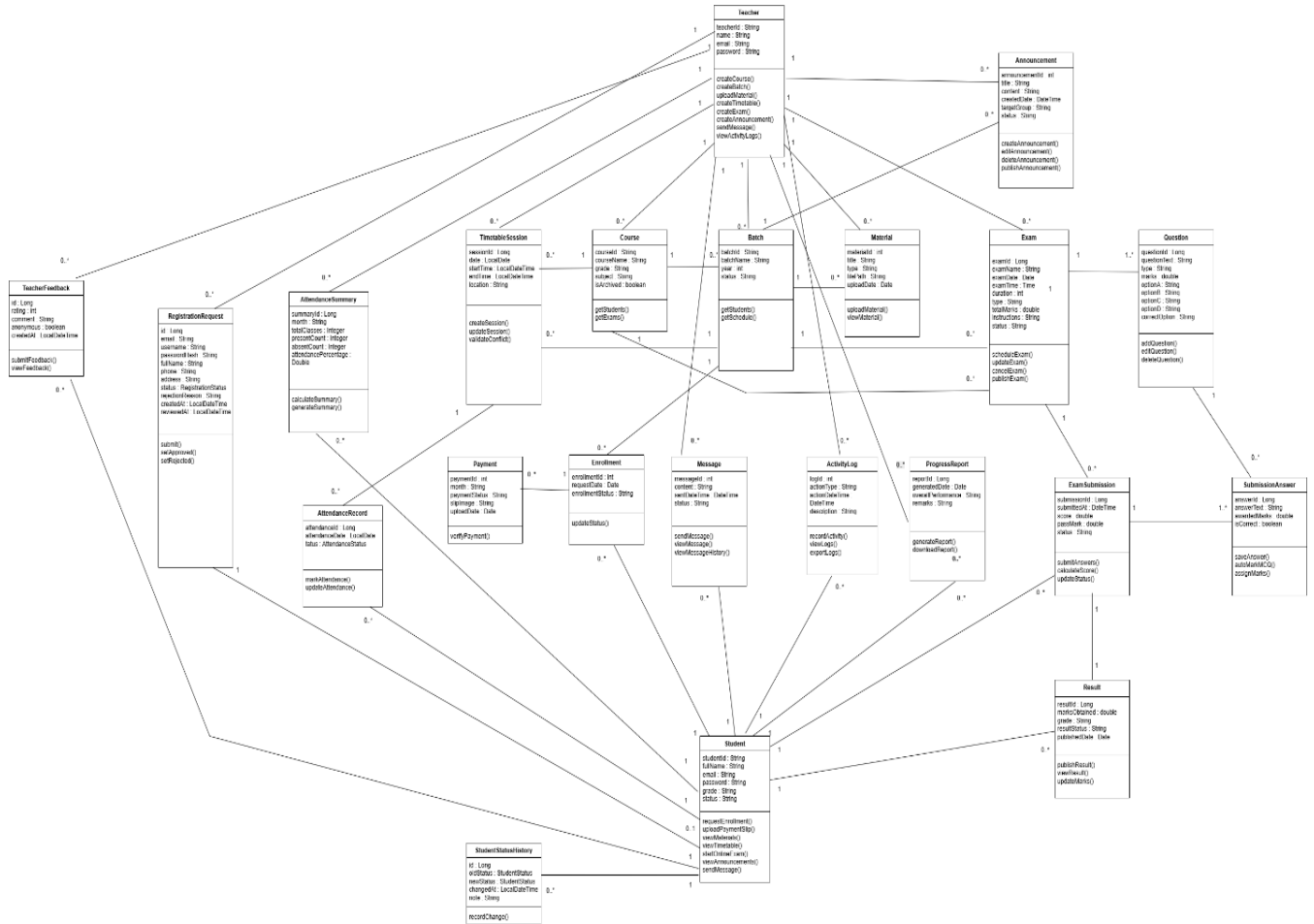


Figure 1: UML Class Diagram

Sequence Diagram:

Approve Student Registration Request

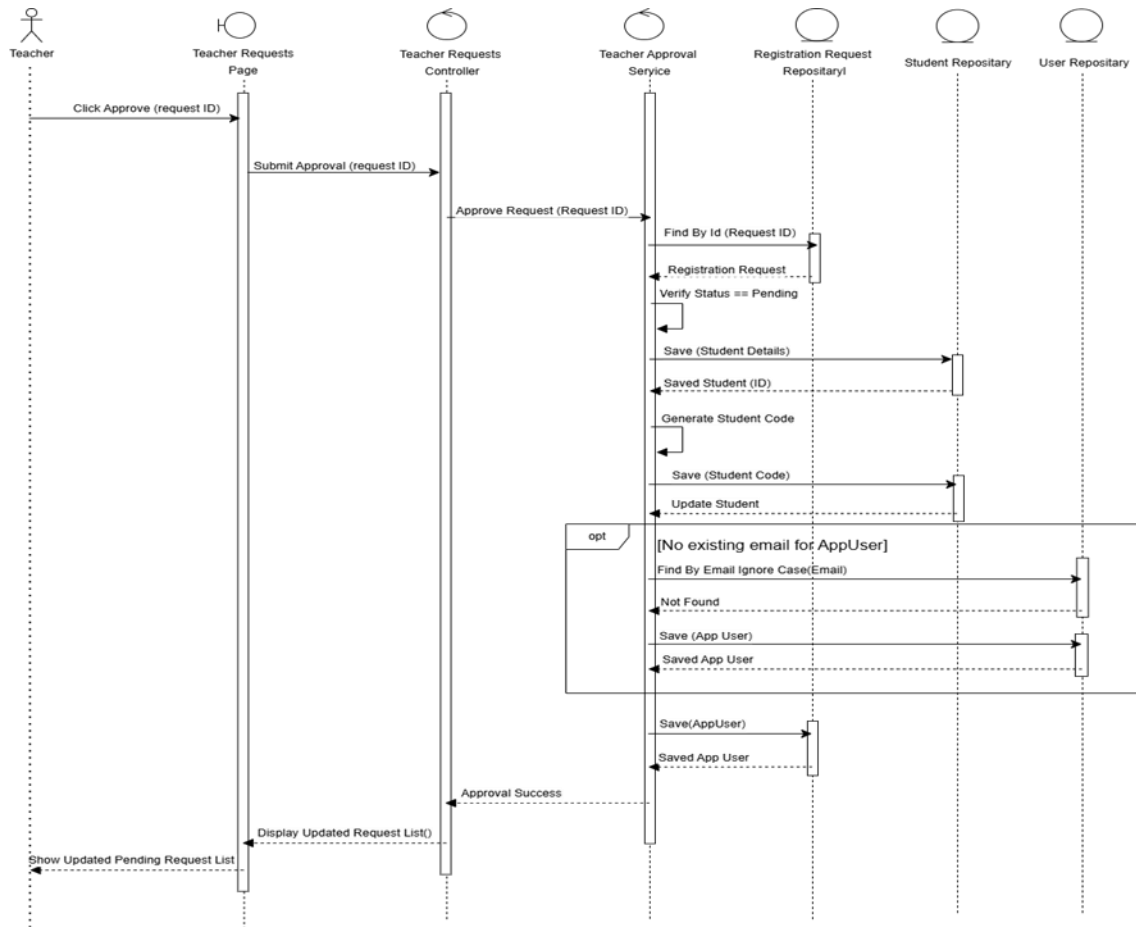


Figure 2: Approve Student Registration Request

Approve or Reject Payment Slips

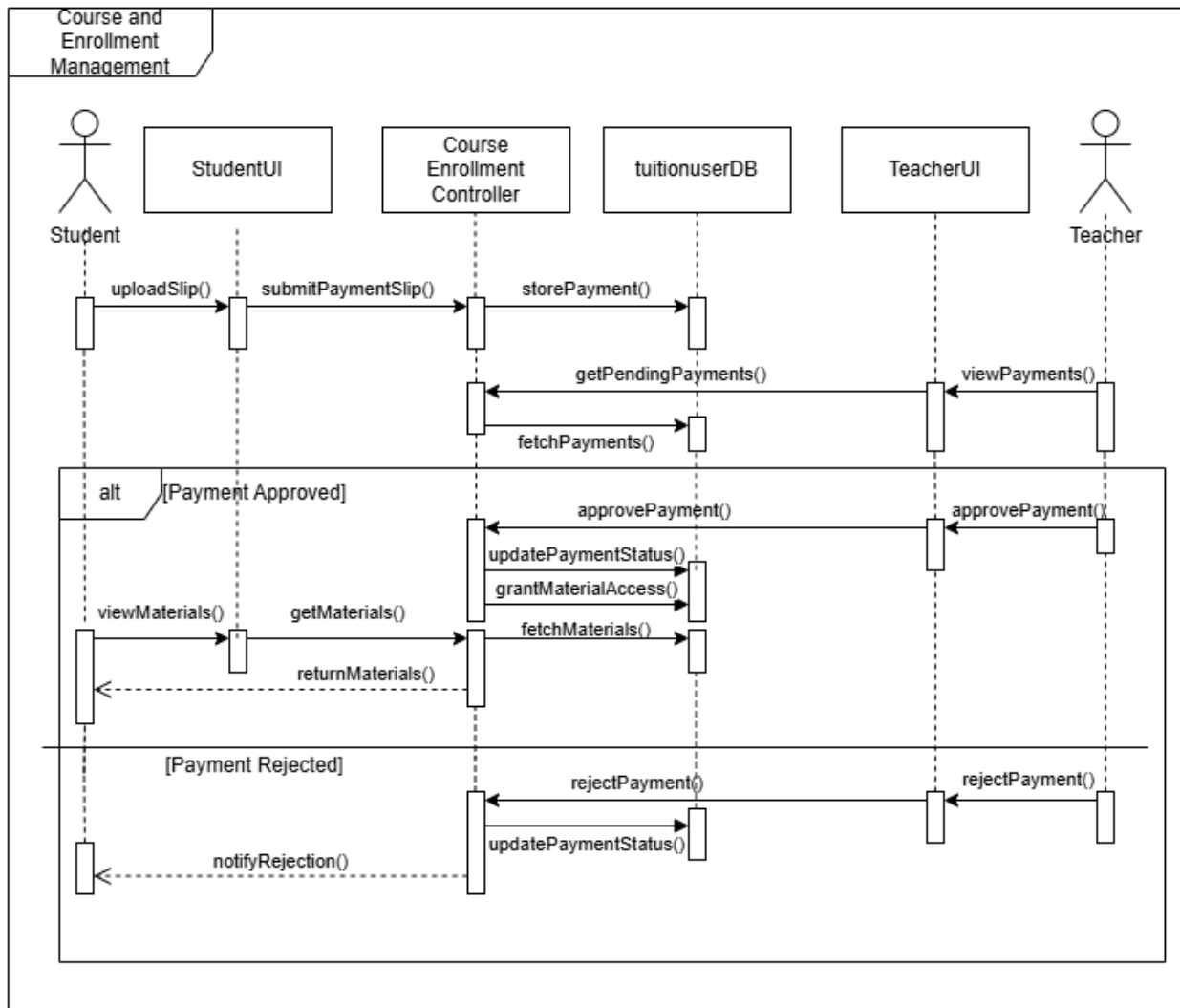


Figure 3: Approve or Reject Payment Slips

Teacher Marks Attendance

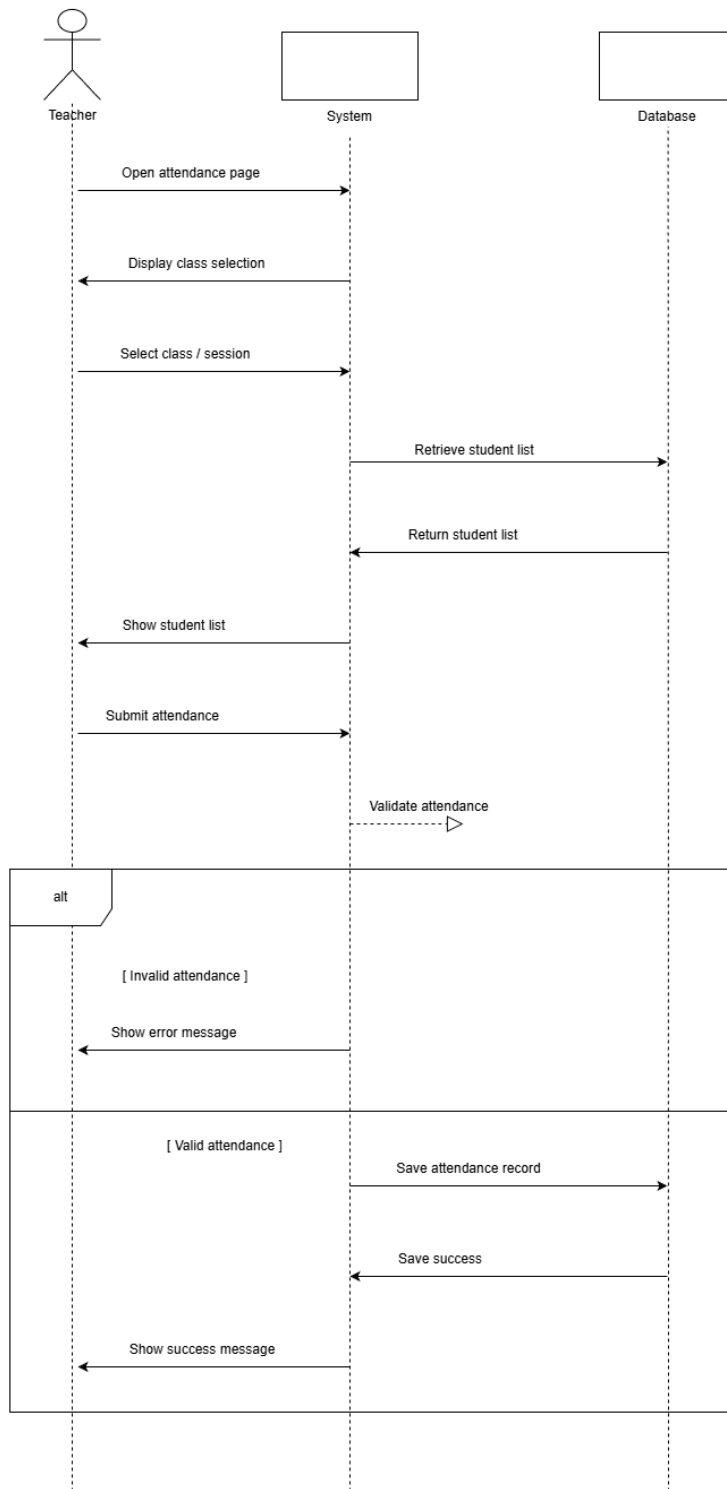


Figure 4:Teacher Marks Attendance

Conduct Online Exam

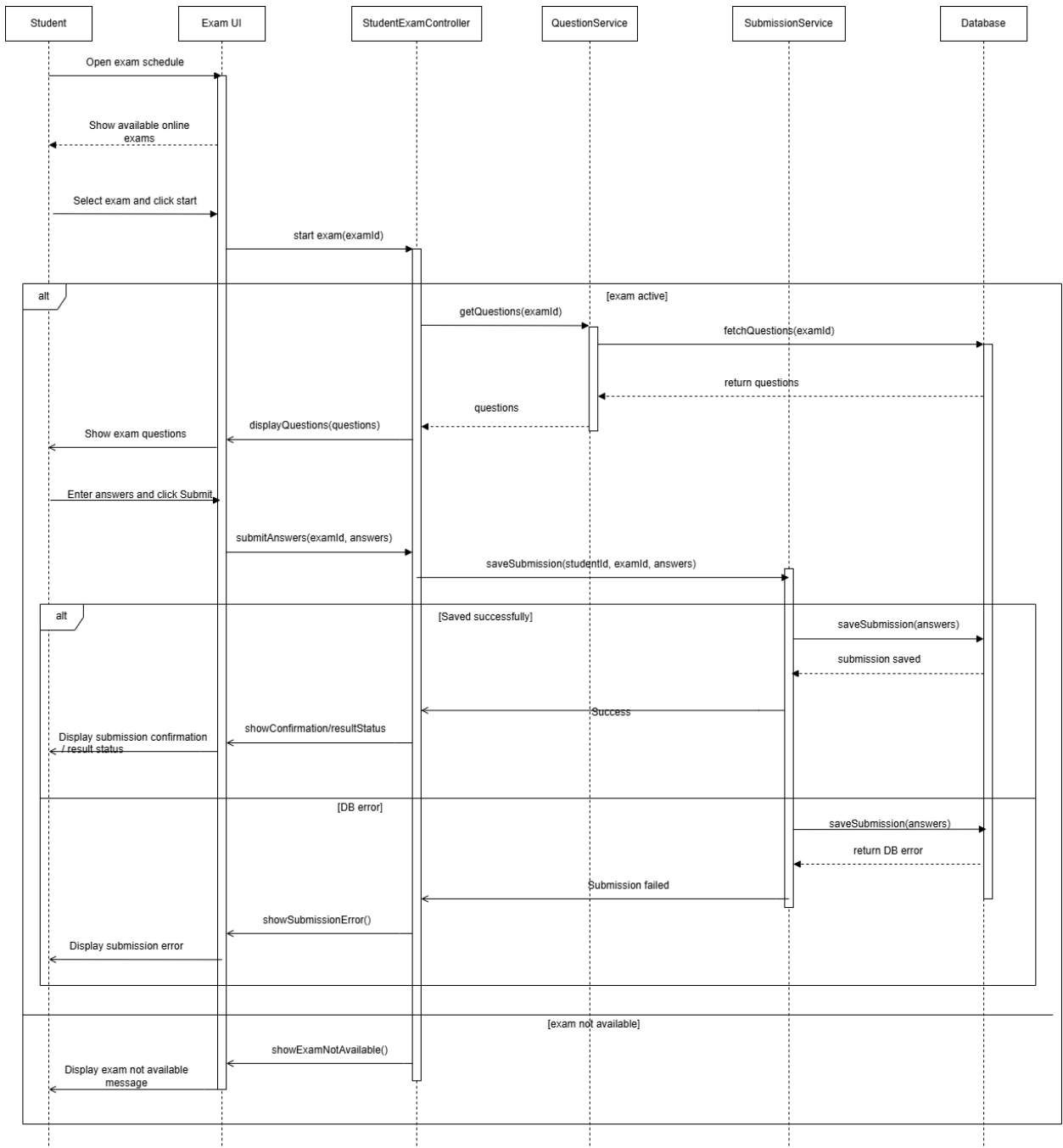


Figure 5: Conduct Online Exam

Activity Diagrams:

Approve Student Registration Request

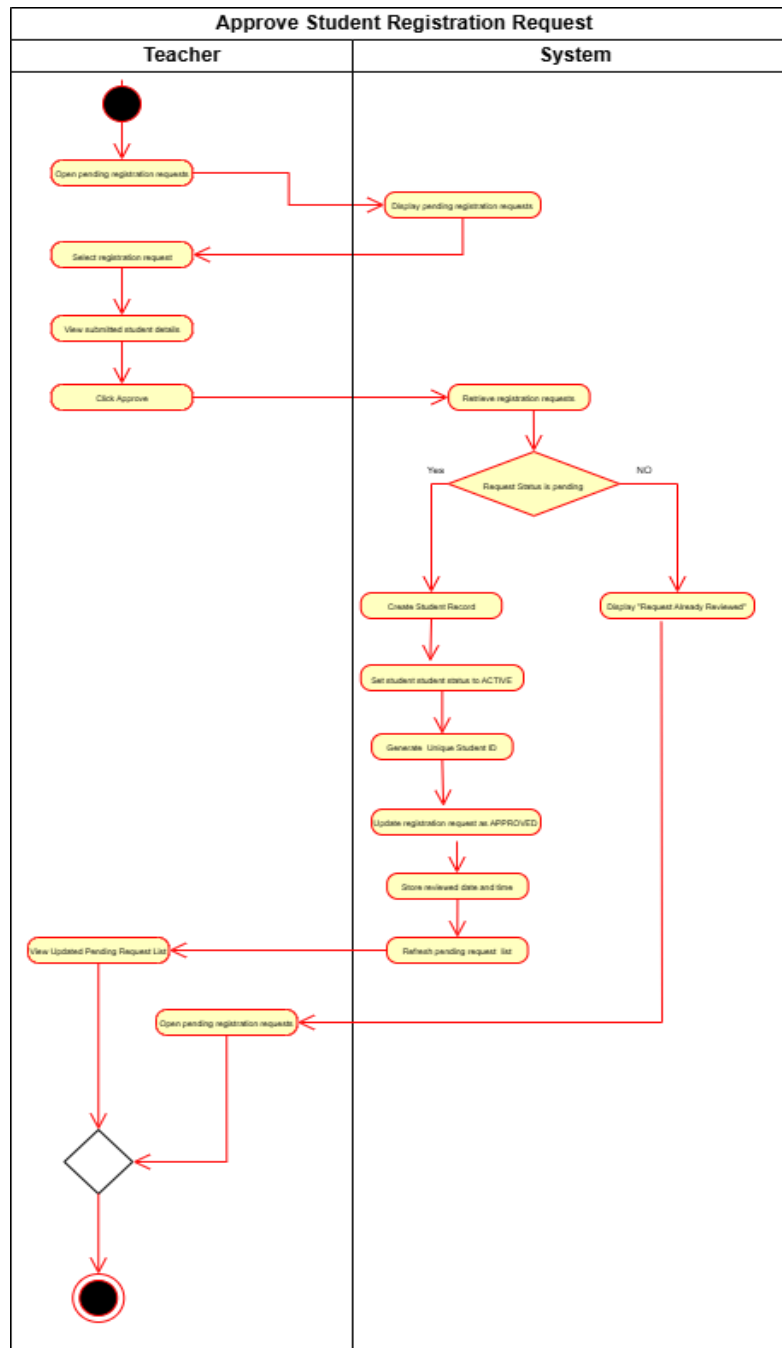


Figure 6: Approve Student Registration Request

Approve or Reject Monthly Payment Slips

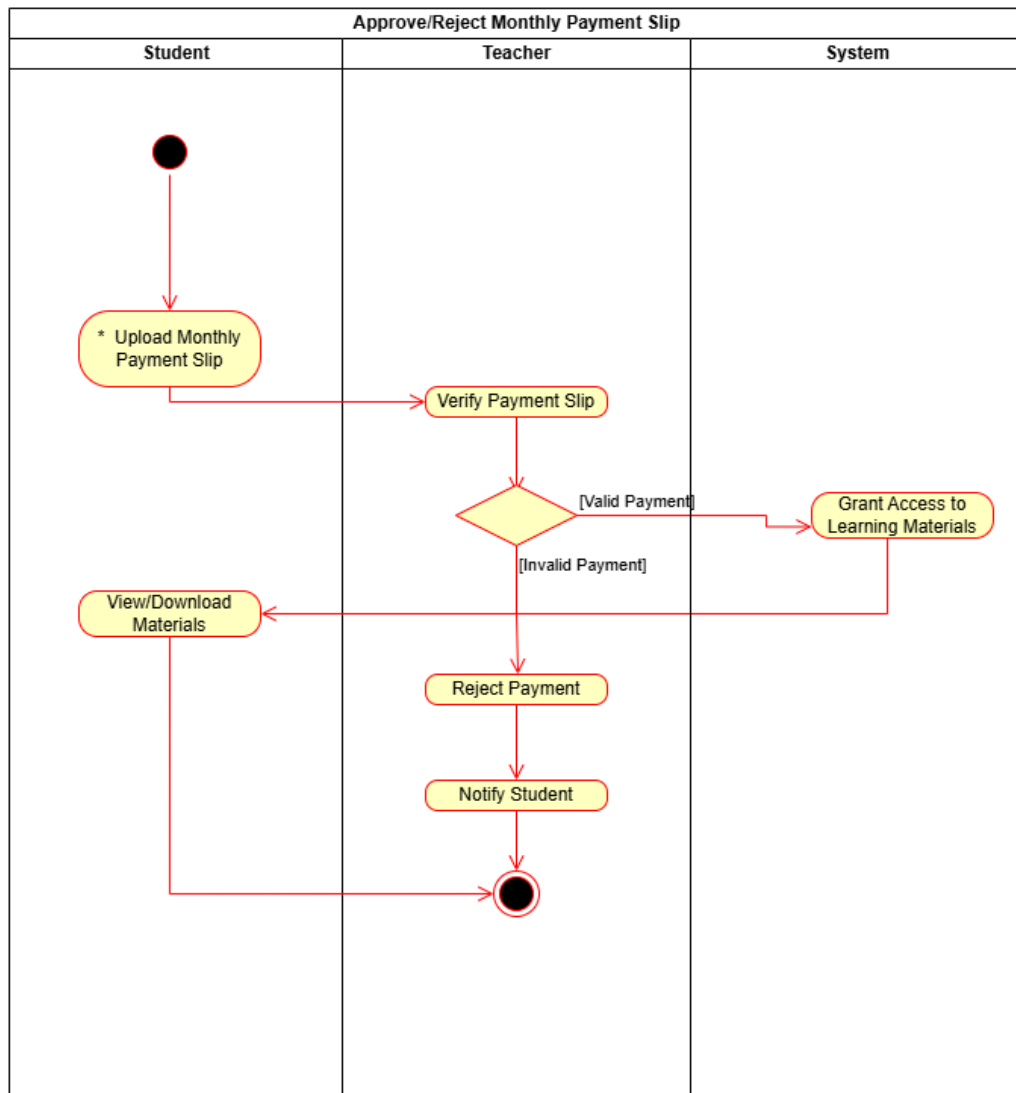


Figure 7: Approve or Reject Monthly Payment Slips

Teacher Marks Attendance

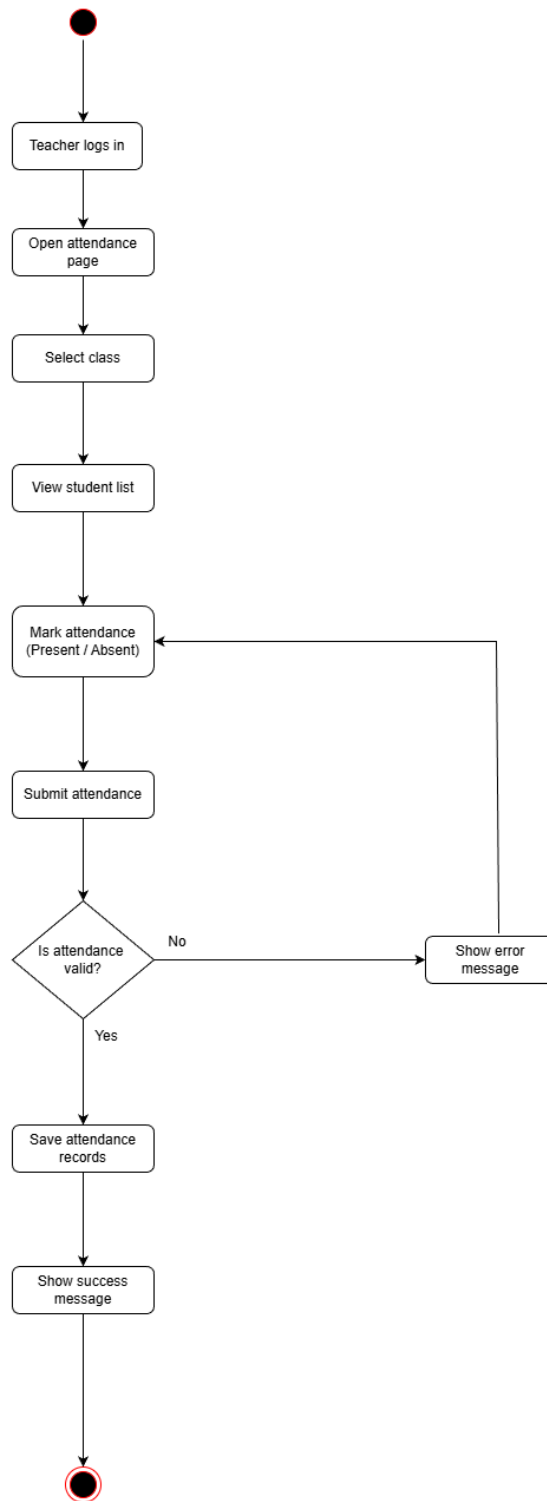


Figure 8:Teacher Marks Attendance

Conduct Online Exam

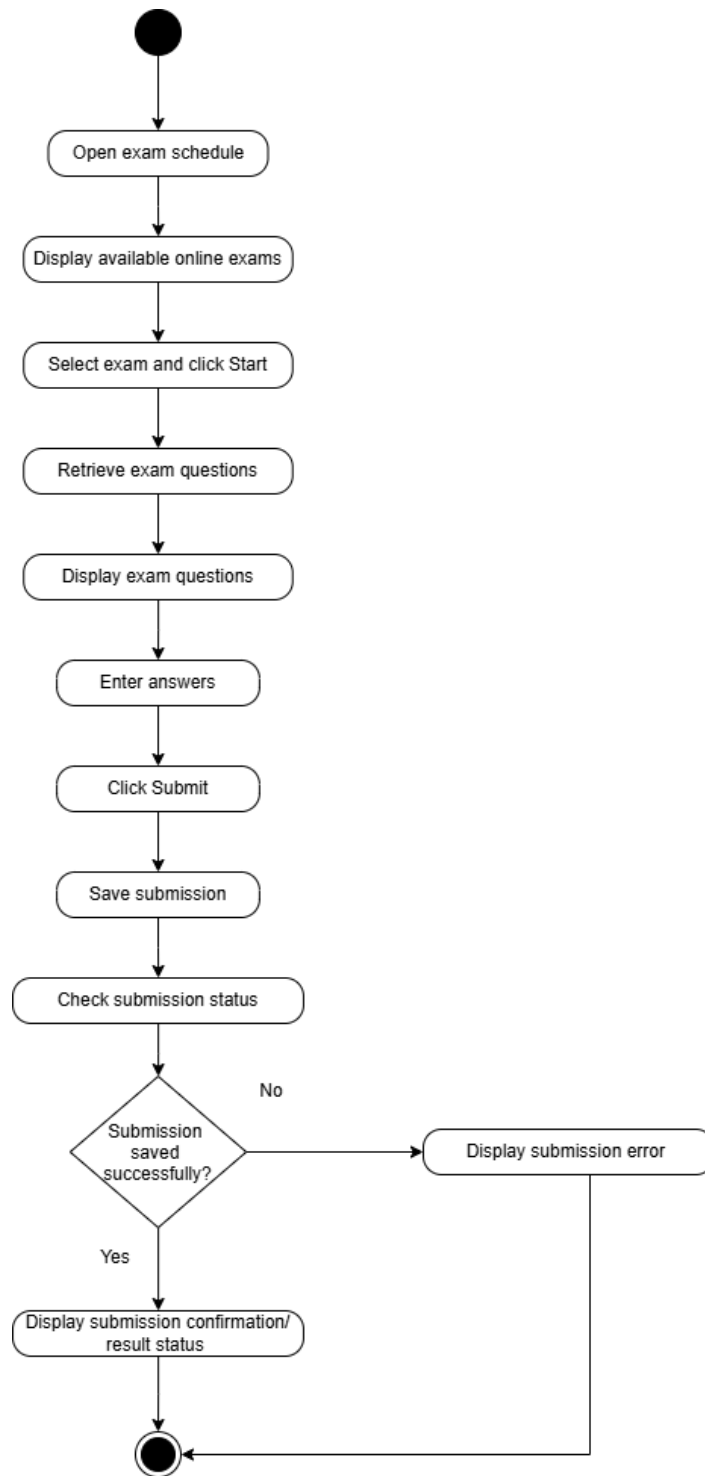


Figure 9: Conduct Online Exam

State Diagram:

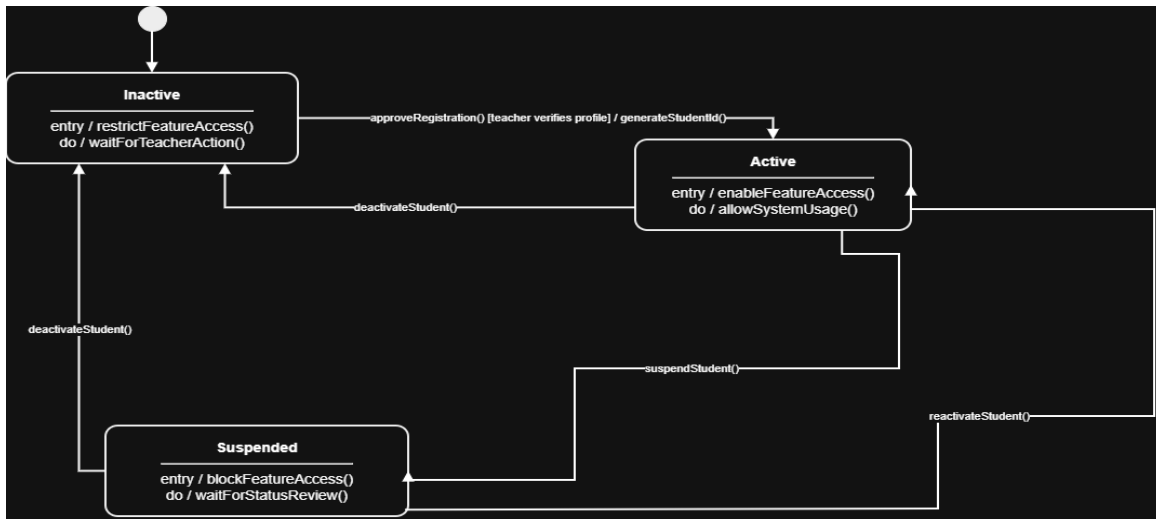


Figure 10: State diagram

UML Use Case Diagram:

Student Management

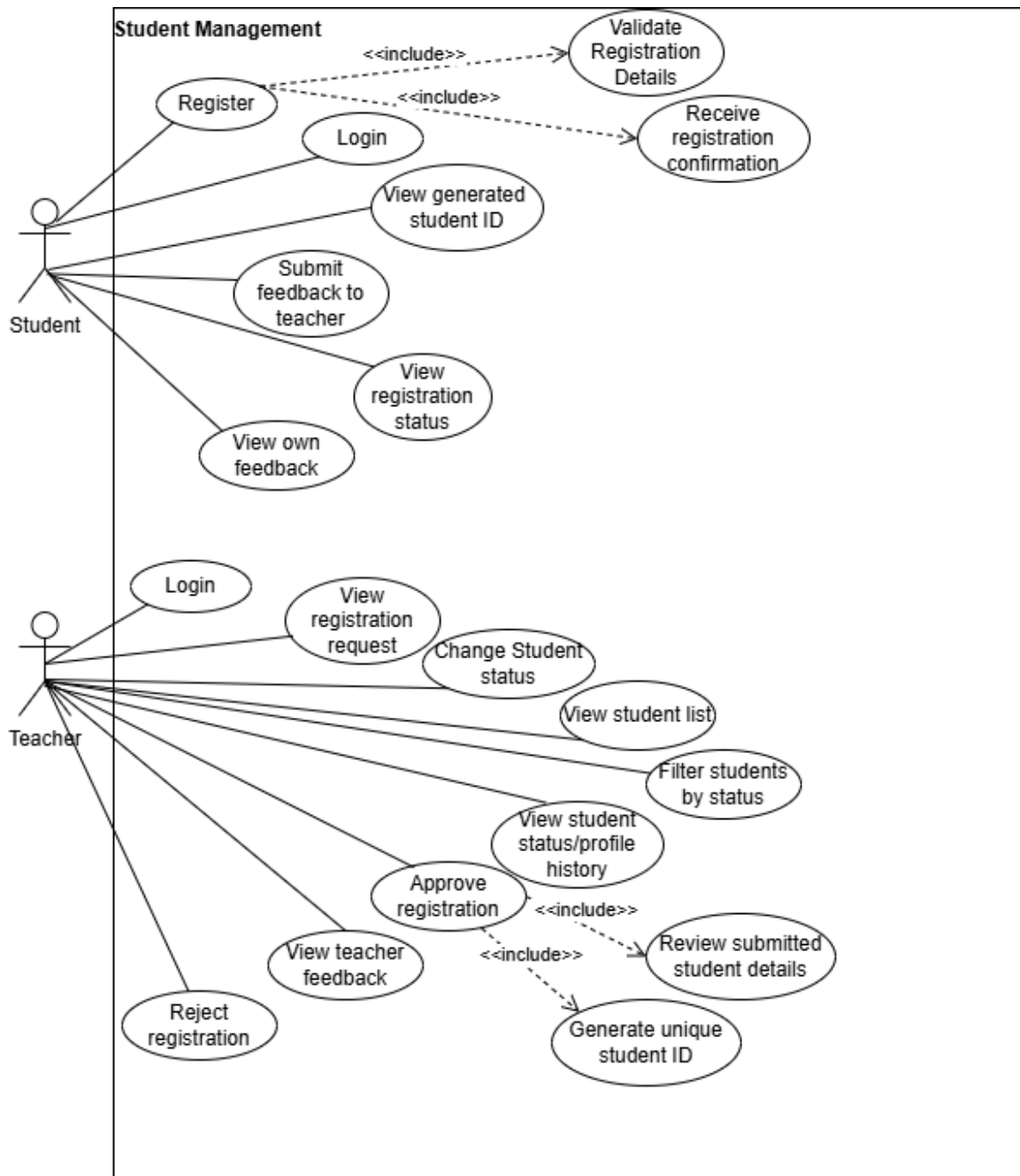


Figure 11: Student Management

Course and Enrollment Management

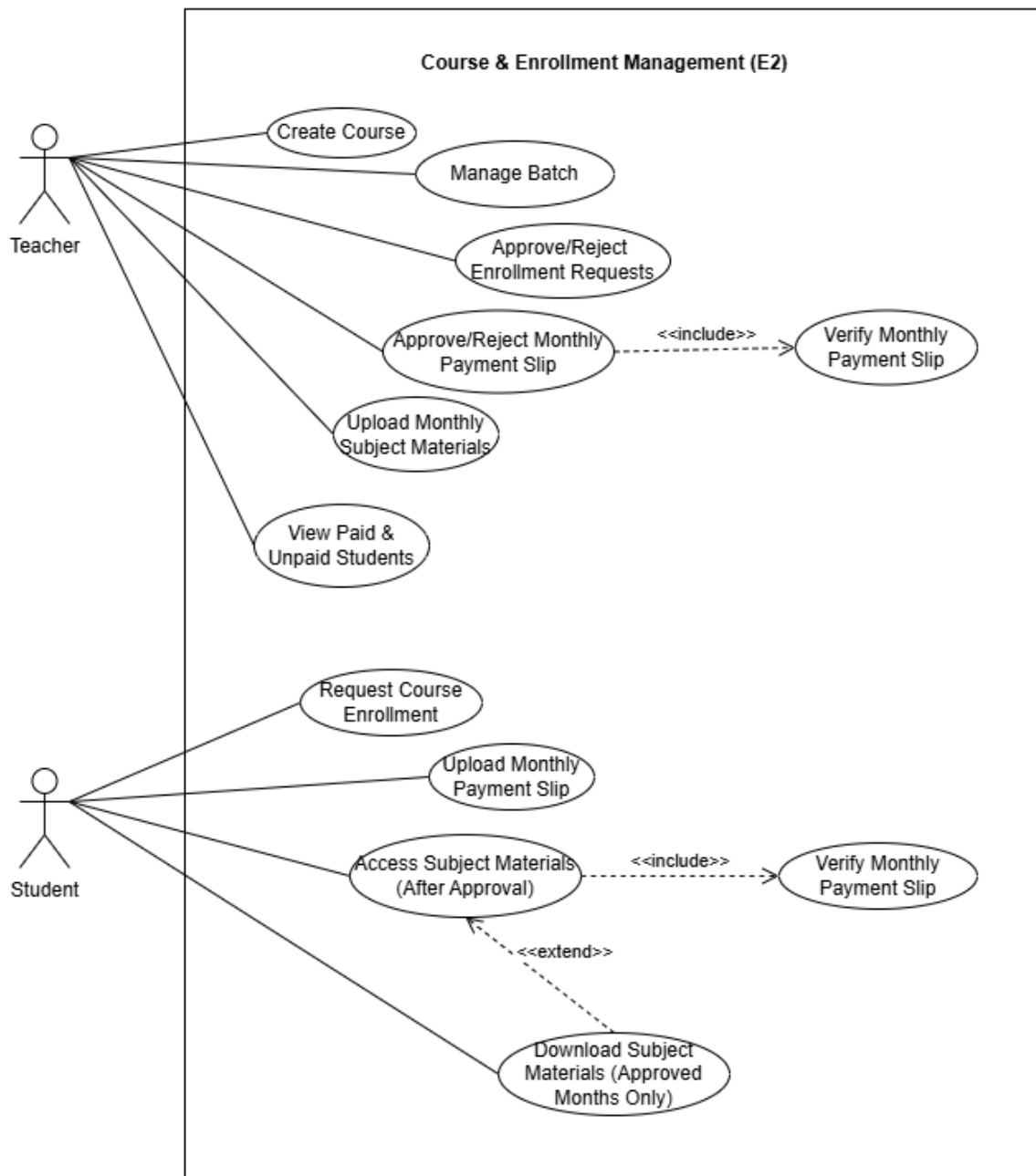


Figure 12: Course and Enrollment Management

Class and Attendance Management

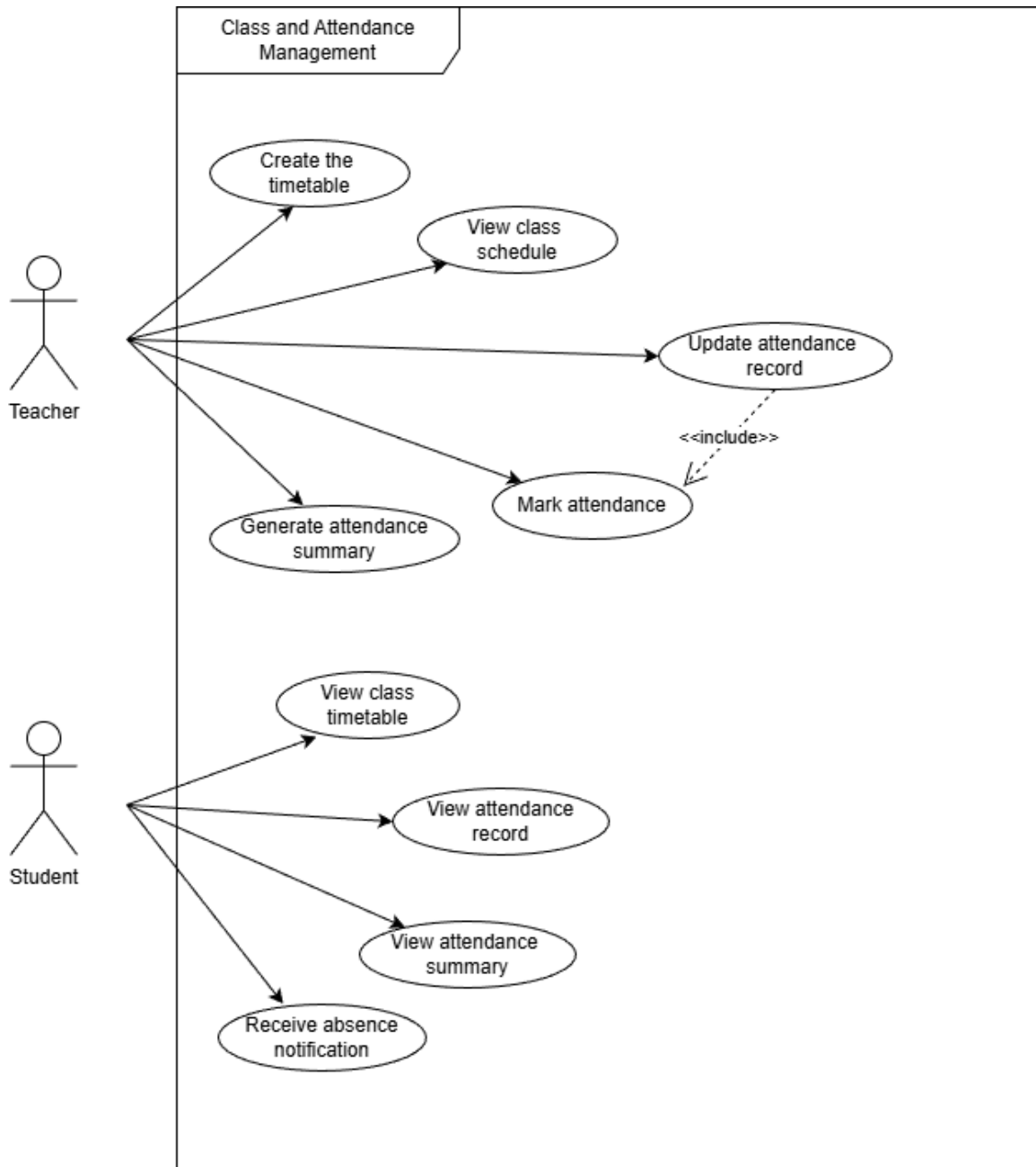


Figure 13: Class and Attendance Management

Examination and Academic Performance Management

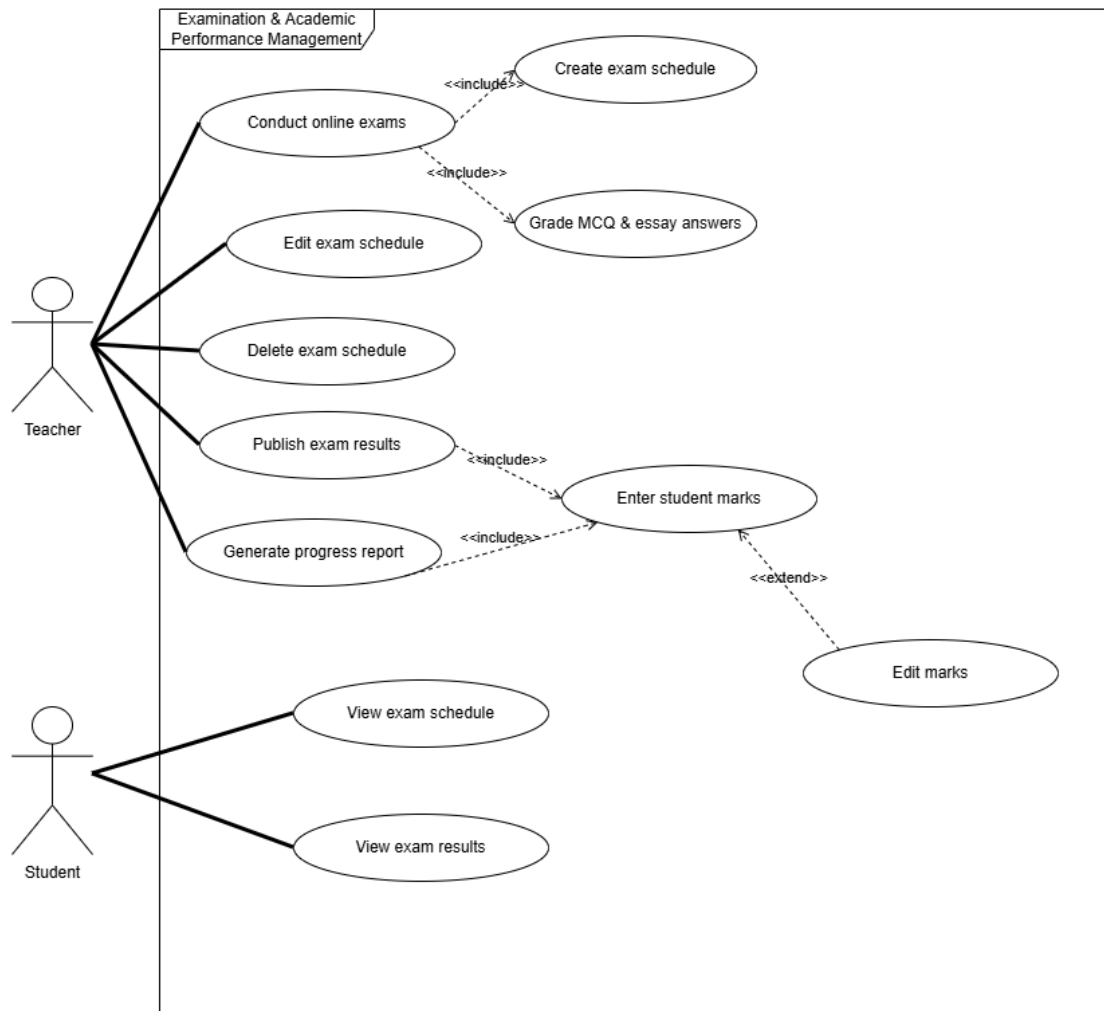


Figure 14: Examination and Academic Performance Management

Academic Communication & Activity Management

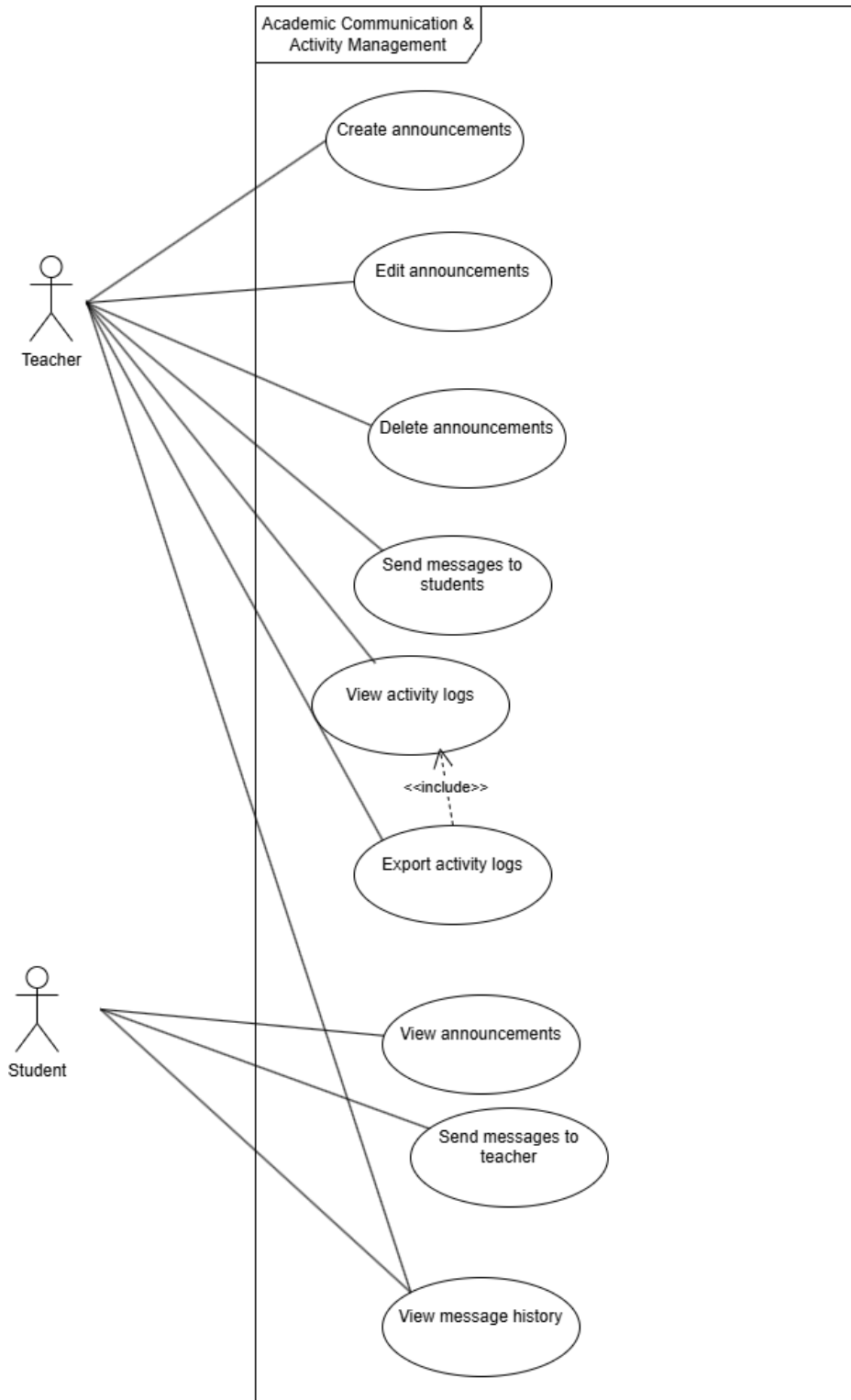


Figure 15: Academic Communication & Activity Management

ER Diagram

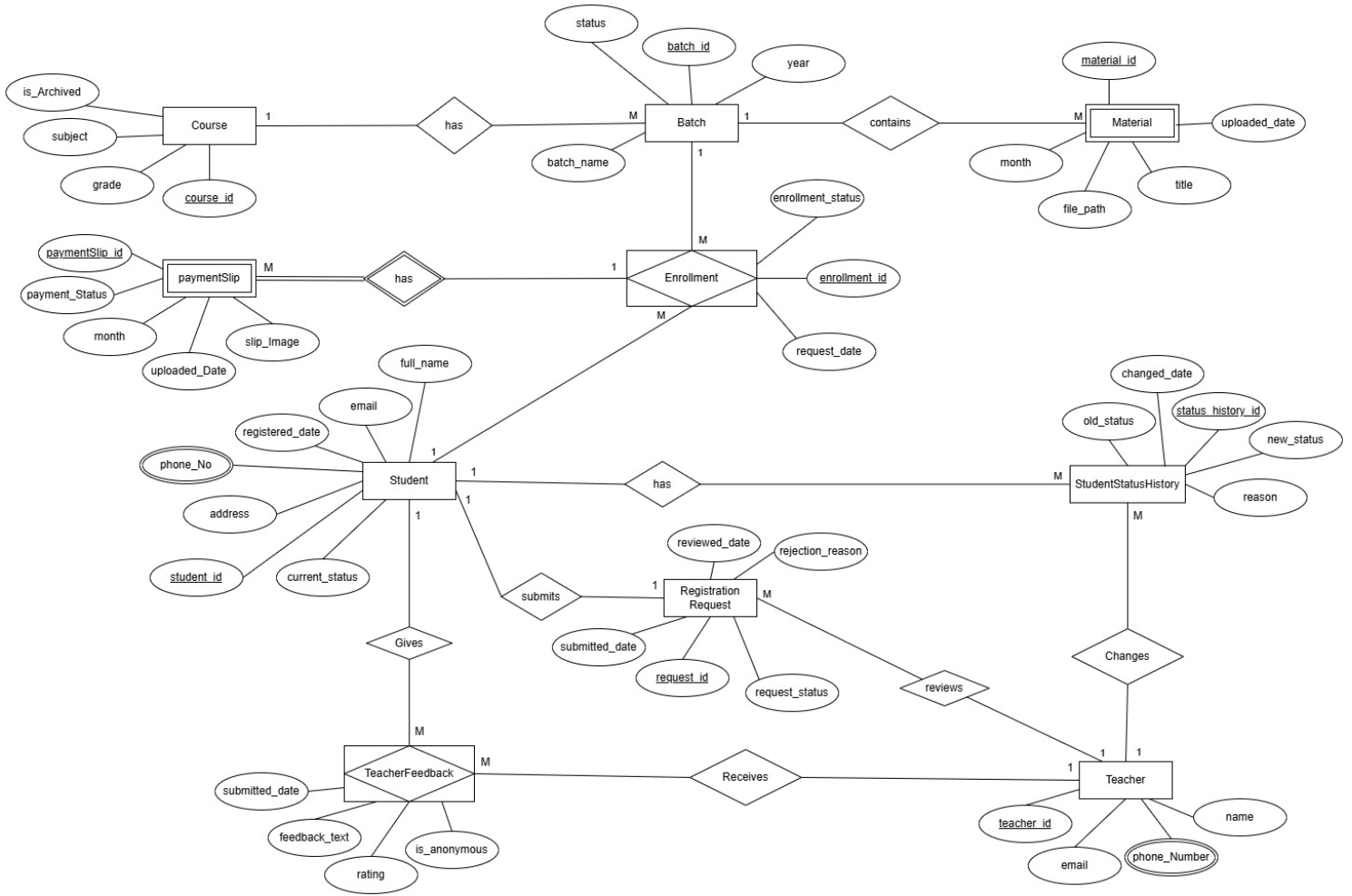


Figure 16: ER Diagram

Appendix B: Test Results

Test ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC01	Student Management	Submit registration form with valid details	Student registration should be saved successfully	Registration saved successfully	Pass
TC02	Student Management	Submit registration form with empty required fields	Validation messages should be displayed	Validation message displayed correctly	Pass
TC03	Student Management	Teacher approves a student registration request	Student status should be updated and student ID generated	Status updated and student ID generated successfully	Pass
TC04	Course and Enrollment Management	Student submits enrollment requests for a course	Enrollment request should be saved successfully	Enrollment request saved successfully	Pass
TC05	Course and Enrollment Management	Teacher approves enrollment request	Students should be enrolled in the selected course/batch	Enrollment approved successfully	Pass
TC06	Course and Enrollment Management	Student uploads payment slip	Payment slip should be uploaded and saved successfully	Payment slip uploaded successfully	Pass
TC07	Class and Attendance Management	Teacher creates a class timetable	Timetable record should be saved and displayed	Timetable created successfully	Pass
TC08	Class and Attendance Management	Student views own class timetable	Relevant timetable should be displayed correctly	Timetable displayed successfully	Pass
TC09	Class and Attendance Management	Teacher marks attendance for a class	Attendance should be recorded successfully	Attendance recorded successfully	Pass
TC10	Examination and Academic Performance Management	Teacher creates an exam schedule	Exam should be saved and displayed in exam list	Exam created successfully	Pass
TC11	Examination and Academic Performance Management	Teacher edits an existing exam schedule	Updated exam details should be saved	Exam updated successfully	Pass
TC12	Examination and Academic Performance Management	Student views exam schedule	Relevant exam schedule should be displayed	Exam schedule displayed successfully	Pass

TC13	Academic Communication and Activity Management	Teacher creates an announcement	Announcement should be saved and displayed	Announcement created successfully	Pass
TC14	Academic Communication and Activity Management	Student views announcements	Announcements should be displayed correctly	Announcements displayed successfully	Pass
TC15	Academic Communication and Activity Management	Teacher sends messages to student	Message should be delivered and saved in history	Message sent successfully	Pass
TC16	Academic Communication and Activity Management	Student views message history	Previous messages should be displayed correctly	Message history displayed successfully	Pass

Table 4: Test Cases

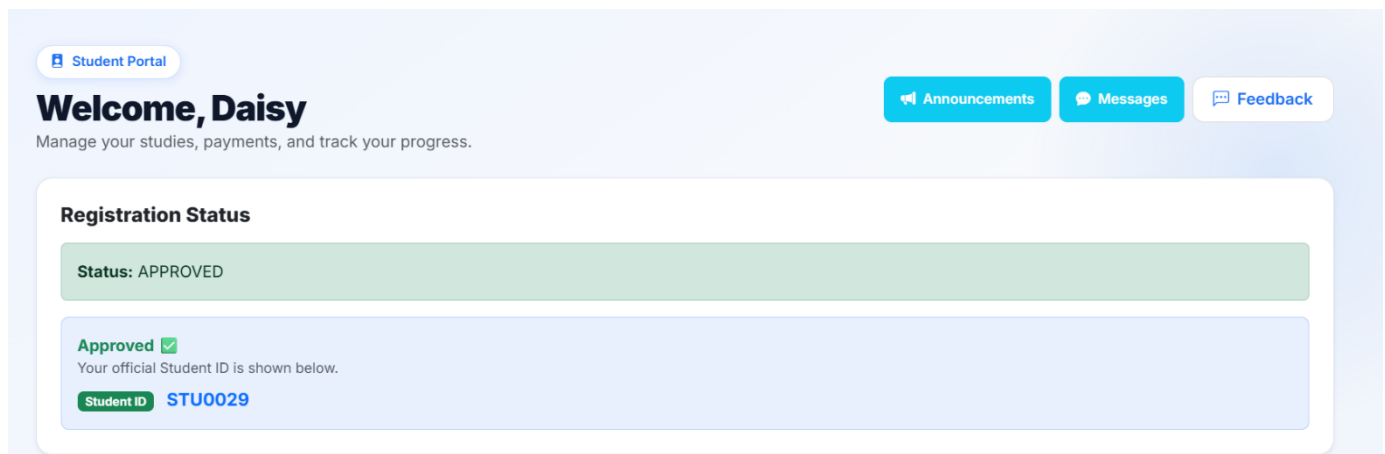


Figure 17: Student Registration Status Interface

Teacher Dashboard
Logout

Academic Catalog

Course Management

Master list of all subjects, grades, and active course cycles.

Manage Batches

Register New Course

COURSE ID

SUBJECT NAME

GRADE LEVEL

ACADEMIC YEAR

Eg: SC10

Eg: Science

Eg: Grade 10

2026

✓

Active & Archived Courses

4 Total

CODE	SUBJECT	GRADE	YEAR	STATUS	MANAGEMENT
MA01	Mathematics	Grade 10	2026	Active	
MA02	Mathematics	Grade 11	2026	Active	
SC01	Science	Grade 10	2026	Active	
SC02	Science	Grade 11	2026	Active	

Figure 18: Course Management Interface

Teacher Dashboard
Logout

Create New Timetable Session

Set the class details, time, and location for this session.

GRADE / LEVEL

SUBJECT

Select Grade

Select Subject

BATCH

NUMBER OF MEMBERS

CLASS TYPE

Select Batch

Waiting for batch...

Detecting...

SESSION DATE

START TIME

END TIME

mm/dd/yyyy

--:--

--:--

LOCATION

e.g. Hall A, Online

Schedule Session

Reset

Cancel

Figure 19: Timetable Session Creation Interface

TMS

Teacher Dashboard Logout

New Exam

Schedule a New Exam

Fill in the details below to notify students about a new assessment.

EXAM NAME

SUBJECT

BATCH / GRADE

EXAM DATE

START TIME

DURATION (HOURS)

LOCATION / TYPE

Type 'Online' to enable question management for this exam.

INSTRUCTIONS (OPTIONAL)

Figure 20: Exam Scheduling Interface

TMS

Teacher Dashboard Logout

Communications

Add Announcement

Fill in the details below and save.

Announcement Details

Title

Content

Target Group

Figure 21: Announcement Creation Interface

Appendix C: Selected Code Listings

Student Management- Student Approval and ID Generation

```
public void approveRequest(Long requestId) { 1 usage
    RegistrationRequest req = requestRepository.findById(requestId)
        .orElseThrow(() -> new RuntimeException("Request not found"));

    if (req.getStatus() != RegistrationStatus.PENDING) {
        throw new RuntimeException("Request already reviewed");
    }

    // Create Student
    Student student = new Student();
    student.setFullName(req.getFullName());
    student.setEmail(req.getEmail());
    student.setUsername(req.getUsername());
    student.setPasswordHash(req.getPasswordHash());
    student.setPhone(req.getPhone());
    student.setAddress(req.getAddress());
    student.setTeacher(req.getTeacher());
    student.setStatus(StudentStatus.ACTIVE);

    studentRepository.save(student);

    // Generate Student Code
    String studentCode = "STU" + String.format("%04d", student.getId());
    student.setStudentCode(studentCode);
    studentRepository.save(student);
}
```

Figure 22: Student Approval and ID Generation

```

// Create matching AppUser if not already present (required for attendance roster)
userRepository.findByEmailIgnoreCase(req.getEmail()).orElseGet(() -> {
    AppUser appUser = new AppUser();
    appUser.setEmail(req.getEmail());
    appUser.setName(req.getFullName());
    appUser.setPassword(req.getPasswordHash());
    appUser.setRole(Role.STUDENT);
    return userRepository.save(appUser);
});

// Update request
req.setStatus(RegistrationStatus.APPROVED);
req.setReviewedAt(LocalDate.now());
req.setCreatedStudent(student);

requestRepository.save(req);
}

```

Figure 23: Student Approval and ID Generation

This method handles the approval of pending student registration requests by creating a student account, generating a unique student ID automatically, setting up the corresponding system user account, and updating the status of the registration request accordingly.

Course and Enrollment Management – Request Enrollment

```
// Student sends enrollment request
@PostMapping("/request")
public Enrollment requestEnrollment(@RequestParam Long studentId,
                                     @RequestParam Long batchId) {

    Student student = studentRepository.findById(studentId)
        .orElseThrow(() -> new RuntimeException("Student not found"));

    Batch batch = batchRepository.findById(batchId)
        .orElseThrow(() -> new RuntimeException("Batch not found"));

    Enrollment enrollment = new Enrollment();
    enrollment.setStudent(student);
    enrollment.setBatch(batch);
    enrollment.setRequestDate(LocalDate.now());
    enrollment.setEnrollmentStatus(EnrollmentStatus.PENDING);

    return enrollmentRepository.save(enrollment);
}
```

Figure 24: Request Enrollment

This method creates a new enrollment record after a student request to enroll in a specified batch. It confirms the existence of the student and batch, updates the request date to the current date, gives a PENDING status, and saves the record for future approval.

Class and Attendance Management - Timetable Conflict Detection Algorithm

```
private void validateConflict(TimetableSession newSession) { 1 usage
    if (newSession.getTeacherId() == null ||
        newSession.getDate() == null ||
        newSession.getStartTime() == null ||
        newSession.getEndTime() == null) {
        return;
    }

    if (!newSession.getEndTime().isAfter(newSession.getStartTime())) {
        throw new TimetableConflictException("End time must be later than start time.");
    }

    List<TimetableSession> sameDaySessions = timetableSessionRepository.findByTeacherIdAndDate(
        newSession.getTeacherId(),
        newSession.getDate());

    for (TimetableSession existing : sameDaySessions) {

        if (newSession.getId() != null && newSession.getId().equals(existing.getId())) {
            continue;
        }

        LocalDateTime newStart = newSession.getStartTime();
        LocalDateTime newEnd = newSession.getEndTime();
        LocalDateTime existingStart = existing.getStartTime();
        LocalDateTime existingEnd = existing.getEndTime();

        LocalDateTime newStart = newSession.getStartTime();
        LocalDateTime newEnd = newSession.getEndTime();
        LocalDateTime existingStart = existing.getStartTime();
        LocalDateTime existingEnd = existing.getEndTime();

        boolean overlap = newStart.isBefore(existingEnd) && newEnd.isAfter(existingStart);

        if (overlap) {
            throw new TimetableConflictException(
                "Time conflict detected. Another timetable already exists on this date");
        }
    }
}
```

Figure 25: Timetable Conflict Detection Algorithm

This method validates timetable sessions and forbids multiple class schedules for the same teacher on the same date and time.

Examination and Academic Performance Management- Exam Submission Marking

```
public class TeacherSubmissionController {  
  
    @GetMapping("/{submissionId}")  
    public String markSubmission(@PathVariable("submissionId") Long submissionId, Model model)  
    {  
        ExamSubmission submission = examSubmissionService.findById(submissionId);  
        List<SubmissionAnswer> answers = submissionAnswerRepository.findBySubmissionId(submissionId);  
  
        model.addAttribute("submission", submission);  
        model.addAttribute("answers", answers);  
  
        return "teacher-mark-submission";  
    }  
  
    @PostMapping("/{submissionId}")  
    public String saveMarks(@PathVariable("submissionId") Long submissionId,  
        @RequestParam("answerIds") List<Long> answerIds,  
        @RequestParam(value = "awardedMarks", required = false) List<Integer> awardedMarks,  
        @RequestParam(value = "teacherFeedback", required = false) List<String> teacherFeedback,  
        HttpSession session) {  
        ExamSubmission submission = examSubmissionService.findById(submissionId);  
        List<SubmissionAnswer> answers = submissionAnswerRepository.findBySubmissionId(submissionId);  
  
        int totalScore = 0;  
        int totalMarks = 0;  
        boolean hasPendingEssay = false;  
  
        for (SubmissionAnswer ans : answers) {  
            totalMarks += ans.getQuestion().getMarks() != null ? ans.getQuestion().getMarks() : 0;  
        }  
    }  
}
```

Figure 26: Exam Submission Marking

This controller method manages the teacher evaluation of student exam submissions. It retrieves submitted answers, processes granted marks and feedback and computes the overall exam result.

Academic Communication and Activity Management– Announcement Creation

```
@PostMapping("/add")
public String add(@Valid @ModelAttribute("announcement") Announcement announcement,
                  BindingResult br, HttpSession session) {
    Long teacherId = (Long) session.getAttribute("teacherId");
    if (teacherId == null) {
        return "redirect:/teacher/login";
    }

    if (br.hasErrors()) return "e5/announcements/form";

    announcement.setTeacherId(teacherId);
    service.create(announcement);
    return "redirect:/e5/announcements/teacher";
}
```

Figure 27: Announcement Creation

This controller method enables teachers to make announcements after confirming the session and inputting information. The announcement is linked to the logged-in teacher and recorded in the system database.